

StudyBase

CS4347 Database Systems

Professor Jalal Omer

Mohit Kansara, Ajay Alluri, Adarsh Goura, Lalith Vennapusa, Anish Patel

December 5, 2025

2. Table of Contents

StudyBase.....	1
2. Table of Contents.....	2
3. Introduction.....	3
4. System Requirements.....	4
4.1 Context Diagram (System Overview).....	4
4.2 Interface Requirements.....	4
4.3 Functional Requirements.....	5
4.4 Non-Functional Requirements.....	6
5. Conceptual Design of the Database.....	7
5.1 ER Model.....	7
5.2 Data Dictionary.....	8
5.3 Business Rules.....	8
6. Logical Database Schema.....	9
6.1 Relational Schema.....	9
6.2 SQL Code.....	9
6.3 Expected Operations & Data Volume.....	11
7. Functional Dependencies and Database Normalization.....	12
7.1 Functional Dependencies and Database Normalization Introduction.....	12
7.1 Study_Groups.....	12
7.2 The Study_Group_Participant Relation.....	12
7.3 Users Relation.....	12
7.4 Enrolled_Courses Relation.....	13
7.5 Courses Relation.....	13
7.6 Sections Relation.....	13
7.7 Location Relation.....	13
7.8 Functional Dependencies and Database Normalization Conclusion.....	13
8. The Database System.....	14
8.1 Database Overview.....	14
8.2 Installation & Requirements.....	14
8.3 Database System Overview.....	14
8.4 Conclusion.....	15
10. User Application Interface.....	15
10.1 Student/User Interface.....	15
10.2 Administrator Interface.....	15
10.3 Interaction.....	15

11. Conclusions and Future Work.....	16
12. References.....	16

3. Introduction

StudyBase is a database-driven application designed to help University of Texas at Dallas students organize and participate in group study sessions. Since the availability and structure of study groups frequently change based on class schedules, deadlines, and student availability, it can be challenging for students to find active study groups or monitor existing ones. StudyBase addresses this problem by storing and managing information about users, courses, locations, and study sessions in a centralized relational database. Through the application interface, students can browse available sessions, join or leave groups, and create their own study groups, while administrators ensure that inactive or inaccurate information is removed.

The rest of this report is organized as follows:

Section 4 describes the system requirements, including the system context, interfaces, and both functional and non-functional requirements.

Section 5 covers the conceptual design of the database, including the Entity-Relationship (ER) diagram and supporting business rules.

Section 6 presents the logical schema derived from the ER model, the SQL schema creation statements, and expected data operations.

Section 7 provides the functional dependency analysis and normalization process for the database relations.

Section 8 describes the implementation of the database system and demonstrates system functionality through screenshots.

Section 10 discusses the design and use of the user application interface.

Section 11 summarizes the project results and outlines future improvements.

Section 12 lists all references used throughout the project.

4. System Requirements

4.1 Context Diagram (System Overview)

StudyBase is a database-driven application that helps UTD students create and join study groups.

External Actors

- Students/Users
 - Search, join, and leave study groups
 - Create and manage hosted sessions
- Administrators
 - Moderate inactive or reported groups
 - Maintain database accuracy

System Components

- StudyBase Front-End Application
 - Web interface for students and administrators
- StudyBase Database (DBMS)
 - Stores all system data: users, groups, locations, courses, etc.

4.2 Interface Requirements

User Interface Requirements

- Login page with role distinction (user/admin)
- Search page with filters
- Group details page showing essential info and participant counts
- Group creation/editing form for hosts
- Join/leave buttons for authenticated users

- User profile page to view/update personal information
- Admin page to review and remove inactive/report-flagged groups

System/Database Interface Requirements

- SQL communication between the application and DBMS
- Appropriate database constraints enforced for consistency
- Optional import of course/location data through SQL or CSV loading

4.3 Functional Requirements

FR-1: User Management

- Register/login users and store profile info in the **Users** table

FR-2: Courses and Enrollments

- Store course and section data
- Track which courses each user is enrolled in

FR-3: Location Management

- Store and manage valid on-campus study locations

FR-4: Study Group Management

- Create, update, and delete groups with:
 - Host, time, location, max/current participants, course (optional)
- Validate group capacity and time constraints

FR-5: Participation

- Users can join/leave groups
- System updates participant count automatically

FR-6: Search and Browse

- Users can filter study groups by course, time, location, and availability

FR-7: Reporting and Cleanup

- Users can report inactive groups
- Admins can remove invalid or empty ones

FR-8: Data Integrity

- DB must enforce all primary key, foreign key, and not-null constraints

4.4 Non-Functional Requirements

1: Performance

- Typical user actions (join/search) should respond within ~2 seconds

2: Reliability

- Regular backups and protection against data loss

3: Security

- Authentication required for modifications
- Secure storage of user data (e.g., email)

4: Usability

- Simple interface with familiar campus/course terminology

5: Scalability

- Support growth in number of users, courses, and groups each semester

6: Maintainability & Extensibility

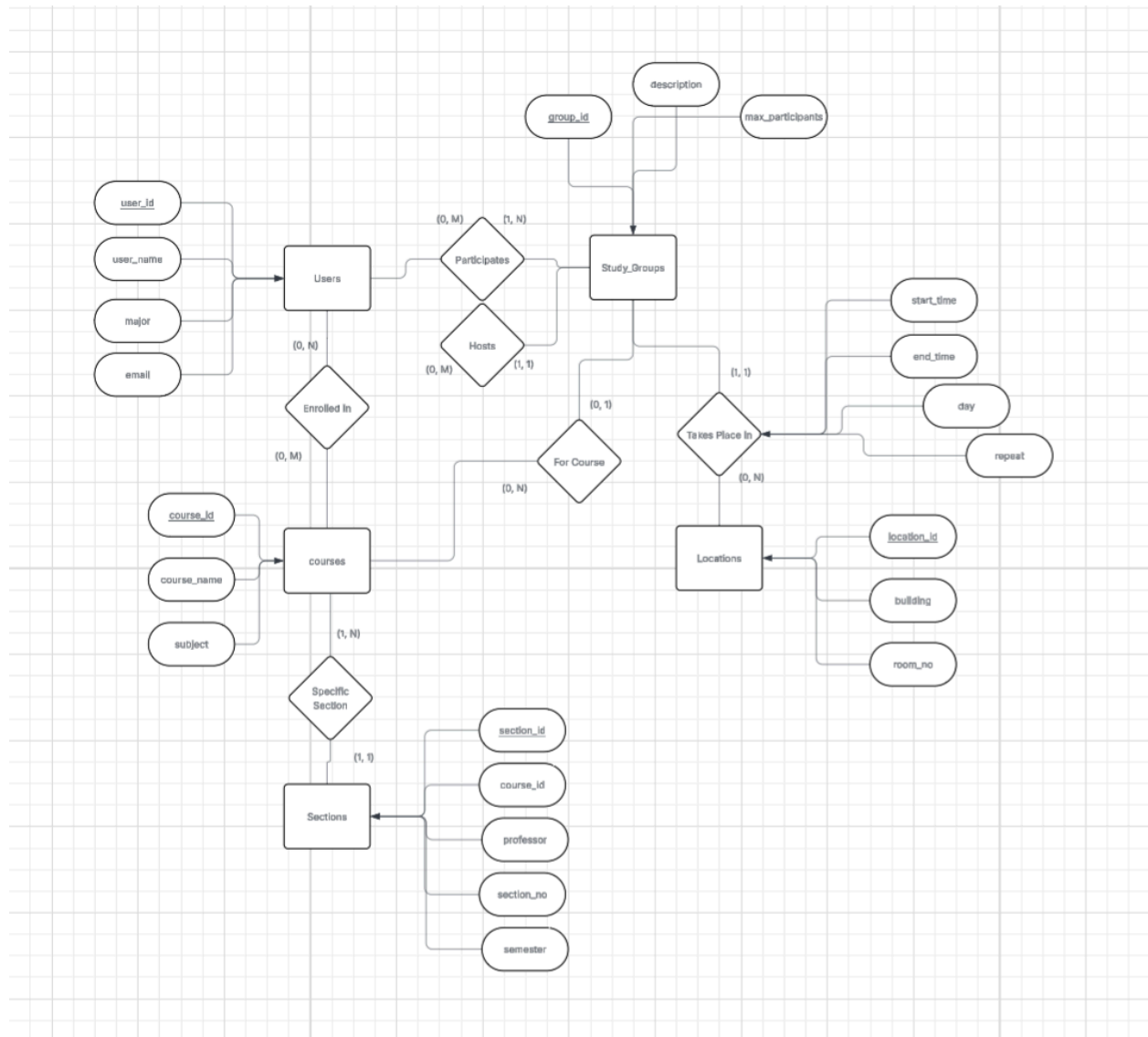
- System structure allows adding new features (e.g., comments, reminders)

7: Portability

- Compatible with common browsers and standard relational DBMS

5. Conceptual Design of the Database

5.1 ER Model



5.2 Data Dictionary

Entity	Attributes	Description
Users	user_id, user_name, major, email	Unique users in the system
Study_Groups	group_id, description, max_participants, start_time, end_time, day, repeat	Hosted study group offerings
Courses	course_id, course_name, subject	Academic courses
Sections	section_id, course_id, professor, section_no, semester	Specific course information
Locations	location_id, building, room_no	Rooms on campus where students meet

5.3 Business Rules

- A study group must have exactly one host (mandatory).
- A study group must have at least one participant.
- Users may participate in multiple study groups.
- Study groups must reference one course.
- If associated, a study group's section must belong to the same referenced course.
- A user may enroll in zero or many courses.
- Sections require a course and cannot exist independently.
- Study groups must take place in a known location.
- start_time must be earlier than end_time.
- max_participants sets a limit on participation capacity.

6. Logical Database Schema

6.1 Relational Schema

M:N relationships are implemented via associative tables:

- Study_Group_Participant for Participates
- Enrolled_Courses for Enrolled In

Table	Keys
Users	user_id PK
Study_Groups	group_id PK, host_id FK, course_id FK, location_id FK
Courses	course_id PK
Sections	section_id PK, course_id FK
Locations	location_id PK
Study_Group_Participant	(user_id, group_id) PK, FK //(pair is primary key and foreign key)
Enrolled_Courses	(user_id, course_id) PK, FK //(pair is primary key and foreign key)

6.2 SQL Code

-- USERS

```
CREATE TABLE Users (  
  user_id CHAR(10) PRIMARY KEY,  
  user_name VARCHAR(30) NOT NULL,  
  major VARCHAR(30),  
  email VARCHAR(50) UNIQUE  
);
```

-- COURSES

```
CREATE TABLE Courses (  
  course_id VARCHAR(10) PRIMARY KEY,  
  course_name VARCHAR(50) NOT NULL,
```

```

    subject VARCHAR(50)
);

-- SECTIONS
CREATE TABLE Sections (
    section_id VARCHAR(10) PRIMARY KEY,
    course_id VARCHAR(10) NOT NULL,
    professor VARCHAR(50),
    section_no VARCHAR(10),
    semester CHAR(4),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- LOCATIONS
CREATE TABLE Locations (
    location_id INT PRIMARY KEY AUTO_INCREMENT,
    building VARCHAR(30) NOT NULL,
    room_no VARCHAR(10) NOT NULL
);

-- STUDY GROUPS
CREATE TABLE Study_Groups (
    group_id INT PRIMARY KEY AUTO_INCREMENT,
    description VARCHAR(200),
    max_participants INT,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    day VARCHAR(10),
    repeat VARCHAR(20),
    host_id CHAR(10) NOT NULL,
    course_id VARCHAR(10) NOT NULL,
    location_id INT NOT NULL,
    FOREIGN KEY (host_id) REFERENCES Users(user_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    FOREIGN KEY (location_id) REFERENCES Locations(location_id),
    CHECK (start_time < end_time)
);

-- PARTICIPATION (M:N)
CREATE TABLE Study_Group_Participant (
    user_id CHAR(10) NOT NULL,
    group_id INT NOT NULL,
    PRIMARY KEY (user_id, group_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)

```

```

        ON DELETE CASCADE,
FOREIGN KEY (group_id) REFERENCES Study_Groups(group_id)
        ON DELETE CASCADE
);

-- ENROLLED IN (M:N)
CREATE TABLE Enrolled_Courses (
    user_id CHAR(10),
    course_id VARCHAR(10),
    PRIMARY KEY (user_id, course_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
        ON DELETE CASCADE
);

```

6.3 Expected Operations & Data Volume

Expected Operations:

Operation Category	Examples	Frequency
User actions	Register, browse groups, join and leave groups	High
Course actions	Enroll, drop or browse sections	Medium
Group actions	Create/edit group or schedule changes	Medium
Admin cleanup	Remove inactive groups	Low

Data Volume:

Table	Estimated Volume	Growth
Users	200-500	Grows every semester
Courses	300-600	Fluctuates/very slow growth
Sections	1000-2000	fluctuates per semester
Group Study Groups	50-200	fluctuates per study group
Participation	900-3000	grows with user activity

7. Functional Dependencies and Database Normalization

7.1 Functional Dependencies and Database Normalization Introduction

The functional dependencies for the StudyBase database were identified based on the semantics of each relation defined in the logical schema. Every table was examined to determine how its attributes depend on the primary key and whether any partial or transitive dependencies exist. The goal was to verify that every relation in the schema satisfies the requirements of Third Normal Form (3NF) and it confirms whether any decompositions were needed.

7.1 Study_Groups

The Study_Groups relation includes a primary key, `group_id`, which uniquely identifies each study group. All other attributes such as the description, maximum and current number of participants, host ID, course ID, location ID, meeting day, meeting times, and repeat flag are fully dependent on this key. This is because there are no dependencies among non-key attributes and because the determinant of each functional dependency is the primary key, this relation meets all criteria for 3NF.

7.2 The Study_Group_Participant Relation

The Study_Group_Participant relation shows the many-to-many relationship between users and study groups. The primary key for it is the composite pair. Since the table contains no additional non-key attributes, the only functional dependencies present are those involving the primary key. As a result the relation contains no partial or transitive dependencies, and therefore it is fully normalized.

7.3 Users Relation

The Users relation uses `user_id` as its primary key, and this key functionally determines the user's major, name, and email address. These dependencies reflect the structure `user_id → major, user_name, email`. Because all non-key attributes depend only on the primary key and

not on each other, relation does not contain transitive dependencies. Therefore, the Users table meets the requirements of 3NF without modifications.

7.4 Enrolled_Courses Relation

The Enrolled_Courses relation also contains a composite primary key composed of `course_id` and `section_id`. The relation contains no additional attributes beyond those participating in the key, meaning the only possible dependencies are those involving the full key. This structure satisfies 3NF, as there are no partial or transitive dependencies.

7.5 Courses Relation

The Courses relation has `course_id` as the primary key, which determines the course name. Since the left hand side of the dependency is a key and there are no additional attributes being able to introduce partial or transitive dependencies, this relation is already in 3NF.

7.6 Sections Relation

The Sections relation uses a composite primary key which consists of `section_id` and `section_of` where `section_of` is a foreign key referencing the Courses table. The attributes `professor` and `semester` are fully functionally dependent on the entire composite key. These dependencies ensure that the relation has no partial dependencies ensure that the relation has no partial dependencies on part of the composite key and no transitive dependencies among non key attributes. Because of this, the Sections table also satisfies 3NF.

7.7 Location Relation

Finally, the Location relation includes `location_id` as its primary key and contains the two descriptive attributes `building` and `room`. Both are fully dependent on `location_id` and do not functionally determine each other. There are no partial or transitive dependencies present in this relation, meaning it is also in 3NF.

7.8 Functional Dependencies and Database Normalization Conclusion

After the evaluation of each relation in the schema, all tables were found to be fully normalized to Third Normal Form. Because each table already meets the requirements of 3NF based on its functional dependencies, no decompositions were necessary. The normalized SQL structures corresponding to these relation appear in the project's `create.sql` file, which has the finalized primary key and foreign key constraints that enforce referential integrity.

8. The Database System

8.1 Database Overview

The StudyBase system consists of a relational database implemented in MySQL and a set of web based interfaces implemented using HTML. This section explains the installation process required to set up the system, the steps for invoking and running it, and an overview of the user interactions demonstrated through the provided interface screens.

8.2 Installation & Requirements

To install the database portion of the system, the user must begin by making sure that a MySQL server is available. This may be accomplished through a standalone installation of MySQL or through an environment such as XAMPP, WAMP, or MAMP. Once the MySQL server is running, the user should open a MySQL client such as MySQL Workbench or a terminal interface. The entire database schema can be created by executing the included create.sql script which contains all CREATE TABLE statements as well as primary key and foreign key constraints. After the schema has been constructed, the user may load sample data by executing the load.sql script. This script imports data from CSV files into each corresponding table in the database. After these steps are complete, the database is fully initialized and ready to support the web based portion of the system.

8.3 Database System Overview

The user interface is built using HTML files that replicate the forms and input designed in earlier project phases. To invoke the system, the user may open the index.html file in any standard browser. This file functions as the home page and provides navigation links to all major features of the system, including registering a user, creating a study group, joining an existing course, and searching for available study groups.

Each interface page demonstrates how users will eventually interact with the underlying database. For example, the Register User page includes form fields for entering a user ID, name, major, and university email. These fields correspond directly to columns within Users table, and upon integration with PHP, will result in the insertion of a new tuple. Similarly, the Create Study Group page gathers the necessary information for the Study_Groups relation, such as the host ID, course ID, day of the week, meeting times, and location. The Join Study Group page collects a participant ID and a group ID, showing the structure of many to many Study_Group_Participant table.

The Enroll in a Course interface allows users to associate themselves with academic courses through the Enrolled_Courses table. The Search Study Groups interface allows users to filter results based on course or day, and the results page provides a structured table meant to display group details returned from database queries once backend integration is completed.

8.4 Conclusion

Together these components demonstrate the intended flow of interaction between users and the database system. Although the current phase emphasizes front-end development and database schema design, the structure of the HTML pages lays a strong foundation for the final phase in which PHP scripts will implement database connectivity, perform CRUD operations, and dynamically update the user interface. The system as designed is modular, scalable, and consistent with the principles of normalized database design established earlier in the project.

10. User Application Interface

The StudyBase user interface was designed as a simple web-based application that allows students and administrators to interact with the study group database. The interface is built using standard web technologies (HTML and a backend with SQL queries) to display information from the database and update it when users perform actions.

10.1 Student/User Interface

- Students use the interface to:
- Search and browse study groups using filters such as course, time, and location.
- View group details including participant counts, schedule, and host information.
- Join or leave groups, which automatically updates the database.
- Create and manage hosted study groups, including editing or deleting their sessions.
- View and update profile information, including their enrolled courses and active sessions.

10.2 Administrator Interface

- Administrators have additional tools to:
- Review all groups in the system.
- Remove or disable inactive or reported groups.
- Manage course and location entries to keep data accurate.

10.3 Interaction

User actions (creating groups, joining groups, etc.) trigger SQL operations through the backend, and database changes are immediately reflected on the interface for real-time feedback.

11. Conclusions and Future Work

The StudyBase project successfully demonstrated how a relational database system can support the dynamic nature of student study groups at the University of Texas at Dallas. By designing and implementing a structured schema with proper integrity constraints, the system ensures accurate tracking of study sessions, users, locations, and course information. The interface and backend logic allow students to easily create, join, and manage study groups, while administrators can oversee the system to maintain data quality. This project helped reinforce key database concepts such as ER modeling, normalization, SQL schema construction, and the importance of user-oriented system requirements.

In the future, the StudyBase system could be expanded with additional features to improve the student experience. Potential enhancements include push/email notifications for upcoming sessions, group chat or messaging capabilities, rating and feedback for hosts, support for recurring sessions with automated scheduling, and integration with UTD authentication systems. Mobile app support and improved location mapping could also make the system more accessible. These enhancements would make StudyBase even more effective in facilitating academic collaboration and improving student success on campus.

12. References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.